

כרטיס ה-LoRa - איפה אנחנו עומדים

רני, 3.9.2026. הקוד ב-github.com/Rani367/e22-lora-arduino

כתבתי את זה כדי שנהיה על אותו עמוד לפני שמתחילים לחלק עבודה. זה מה שהבנתי מהמסמך של הפרויקט, מהמדריך של המודול, ומהתשובות של עדן היום.

מה הפרויקט בעצם

על הלוויין יש כרטיס תקשורת עם מודול LoRa של Ebyte. ה-OBC לא מדבר איתו, רק מדליק ומכבה, אז הכרטיס צריך להסתדר לבד: כל כמה זמן לשדר ביקון עם מונה, ובין לבין להאזין ולשדר חזרה מה שנקלט. אנחנו כותבים את הקוד שרץ עליו, בסביבת ארדואינו.

חילקנו את זה לשניים. החלק הראשון הוא ספרייה שיודעת לדבר עם המודול. החלק השני הוא הלוגיקה של הלוויין (ביקון, משיב, מעבר ביניהם) שתשתמש בספרייה. החלק הראשון כתוב ומתקמפל, אבל עוד לא רץ על חומרה.

מה למדנו על המודול

המסמך המקורי של הפרויקט מדבר על פני SPI, NSS, DIO0 וכו'. זה לא רלוונטי למודול שיש לנו. ה-E22 הוא לא שבב LoRa חשוף אלא לוח קטן עם מיקרו-בקר משלו, ומדברים איתו ב-UART (9600 כבירת מחדל). חוץ מה-UART יש שלושה פינים שחשובים: M0 ו-M1 בוחרים את מצב העבודה, ו-AUX אומר אם המודול עסוק (LOW) או פנוי (HIGH). זהו.

עוד כמה דברים שכדאי לדעת לפני שמתכננים משהו:

- אי אפשר לבחור Spreading Factor. מה שיש זה "קצב אוויר" בין 2.4 ל-62.5 kbps, וכל קצב הוא איזה צירוף SF/BW ש-Ebyte החליטו עליו. הניסוי של "לשנות SF כל 10 שניות" מהמסמך הופך לשינוי קצב אוויר. אותו רעיון.
- הצד השני של הקישור כנראה צריך להיות גם Ebyte. E22 עוטפים את ה-LoRa בפרוטוקול משלהם עם FEC, ולא מצאנו אף אחד שמפענח את זה עם SX1272 רגיל. עדן אמר שדוד קלט פעם עם משהו אחר, אז נבדוק במקום לנחש. בכל מקרה הכרטיס במקור נועד לתקשורת בין לוויינים עם אותו כרטיס.
- התדר הוא מספר ערוץ: 410.125 MHz ועוד ערוץ במגה-הרץ, ערוצים 0 עד 83. ברירת המחדל היא ערוץ 23, כלומר 433.125. הלוויין הראשי על 436.4, אז ערוצים 25 עד 28 מחוץ לתחום. עדן ביקש שהתדר יהיה פרמטר ולא קבוע בקוד.
- המודול לא שומר על גבולות של פקטות בצד ה-UART. שתי פקטות שמגיעות ברצף יכולות לצאת כזרם אחד. פורמט הפקטה שלנו יצטרך בייטים של סנכרון, אורך ו-checksum. לדעתי זו ההחלטה הכי חשובה בחלק השני.

מה עדן ענה

שאלה	תשובה
איזה מודול על הלוח?	E22-400T30D, מסומן V2. לא ה-T22S שחשבנו. אותו מדריך ואותו פרוטוקול, אבל זו גרסת ה-DIP של 1 וואט: 7 פינים, אין RESET, דורש 5V, ומושך עד 620mA בשידור.
הבקר?	ATmega328PB, לא ESP32. לוגיקה של 32KB, 2KB RAM, 5V פלאש. בלי USB על הלוח.
פינים?	ריף שולח סכמה. מהתמונה שראינו המודול יושב בשקע J3, פינים 6 ו-7 הם VCC ו-GND, ו-M0/M1 מגיעים מהבקר דרך נגדים. איזה GPIO בדיוק, עוד לא יודעים.
תדר?	פרמטר. הלוויין על 436.4. רישוי לא נדרש במעבדה בהספקים האלה.
תחנת קרקע?	נבדוק ביחד, ראו למעלה.
קושחה?	נקרא בעצמנו עם AT+FWCODE=?. עדן ביקש שזה יהיה הדבר הראשון שנריץ.
צריבה?	לפי ריף אפשר לחבר את הלוח בכבלים לארדואינו אחר ולהשתמש בו כמתכנת. עוד לא ניסינו.

מה יש בריפו

src/E22.h ו- src/E22.cpp זו הספרייה. היא יודעת לעבור בין מצבים עם M0/M1 ולחכות ל-AUX, לקרוא ולכתוב את פ רגיסטרי התצורה (ואחרי כל כתיבה לקרוא חזרה ולוודא), לשלוח פקודות AT, ולהעביר בייטים הלוך וחזור. בלי תלויות, ומספיק קטנה ל-2KB. כל המתנה בה היא עם timeout, כי אם משהו נתקע על הלוויין אף אחד לא יבוא לשחרר.

יש שלוש דוגמאות ב- examples/ :

- ReadConfig קורא את הרגיסטרים וגרסת הקושחה ומדפיס. זה מה שמריצים ראשון על לוח חדש.
- PingPong לשני לוחות, אחד שולח PING והשני עונה PONG, עם RSSI.
- AirRateSweep הניסוי מהמסמך: שני הלוחות עוברים על ששת הקצבים, 10 שניות כל אחד.

ב- docs/ יש את המדריך של המודול ואת המסמך המקורי. יש גם CI שמקמפל את הדוגמאות ל- ATmega328PB ול-ESP32 בכל push, וכרגע הכל ירוק.

איך מריצים

```
brew install arduino-cli
arduino-cli config init
arduino-cli config add board_manager.additional_urls
https://mcudude.github.io/MiniCore/package_MCUdude_MiniCore_index.json
arduino-cli core update-index
arduino-cli core install MiniCore:avr
git clone https://github.com/Rani367/e22-lora-arduino.git
cd e22-lora-arduino
arduino-cli compile --fqbn MiniCore:avr:328:variant=modelPB,clock=16MHz_external -
-library . examples/ReadConfig
```

אם מישהו מעדיף ESP32 על השולחן, מוסיפים גם את `esp32:esp32` ומתקינים `https://espressif.github.io/arduino-esp32/package_esp32_index.json` (הורדה של בערך 2GB). אפשר גם ב-Arduino IDE, פשוט להעתיק את התיקייה ל-`~/Documents/Arduino/libraries/`.

בכל דוגמה יש בלוק בהתחלה שבוחר פורט ופינים לפי הלוח. המספרים ל-ATmega שם הם ניחוש עד שתגיע הסכמה.

חיווט למודול

פין במודול	מה זה	הערות
M0 1	מצב, ביט 0	יציאה מהבקר. לא להשאיר צף.
M1 2	מצב, ביט 1	אותו דבר.
RXD 3	נתונים אל המודול	כניסה של 3.3V. מה-ATmega (5V) חייב דרך מחלק או level shifter.
TXD 4	נתונים מהמודול	
AUX 5	עסוק/פנוי	רק לקרוא. אם מושכים אותו ל-LOW בזמן הדלקה המודול נכנס למצב עדכון קושחה ומפסיק לדבר.
VCC 6	5V	עד 620mA בשידור. אם הספק צונח המודול מתאפס באמצע פקטה.
GND 7		

ההצלחה הראשונה נראית ככה: ReadConfig מדפיס ערוץ 23 (433.125 MHz), קצב 2.4 kbps, הספק 0 level, ואחריהם גרסת קושחה. זה מה שהמודול יוצא איתו מהמפעל.

הצעה לחלוקת עבודה

שלושה כיוונים שכמעט לא תלויים אחד בשני. מי לוקח מה, נחליט ביחד.

- חומרה.** להשיג את הסכמה מריף, להבין את הפינים ואיך צורבים, לחוות מודול ולהריץ ReadConfig ואז PingPong. לתעד מה עבד ומה לא ולפתוח issues על מה שנשבר בספרייה. זה החלק שהכי חסר עכשיו, כי שום שורת קוד עוד לא רצה על חומרה.
- החלק השני.** להגדיר את פורמט הפקטה (סנכרון, סוג, מונה, זמן פעולה, קצב, אורך, CRC, payload), לכתוב את מכונת המצבים ביקון/משיב על watchdog, millis(), ולשמור את המונה ב-EEPROM כדי שישרוד כיבוי. זה sketch נפרד שמשתמש בספרייה.
- הספרייה והניסוי.** לתקן מה שהבדיקות מעלות, להריץ את AirRateSweep בשטח ולרשום RSSI ואחוזי קליטה לכל קצב. אולי לשאול את Ebyte על הפקודה AT+ULORAM (שליטה ישירה ב-SF/BW, קיימת אבל לא מתועדת).

כמה כללים

- עובדים ב-branch ופותרים PR ל-main.
- לפני push מקמפלים לפחות דוגמה אחת. ה-CI יתפוס את השאר.
- מה שלא רץ על חומרה מסומן ככה ב-README. כשמשו עובד באמת, מעדכנים.
- לא נוגעים בערוצים 25 עד 28.
- בניסויים משנים תצורה עם persist=false כדי לא לשחוק את הפלאש של המודול.
- הספרייה רק מזיזה בייטים. פורמט, checksum ומונים נשארים בסקצ' שמעליה, ככה אותה ספרייה תשמש גם את תחנת הקרקע.
- 2KB RAM. מחרוזות בתוך F(), באפרים קטנים, בלי printf.

מה עושים עכשיו

1. סכמה מריף, לעדכן את הפינים בדוגמאות, להבין את הצריבה.
2. ReadConfig על מודול אמיתי. לראות את תמונת המפעל ואת גרסת הקושחה.
3. PingPong בין שני מודולים, ואז AirRateSweep עם רישום RSSI.
4. רק אז החלק השני.